Food Recipe Recommendation System

Phase 1 Project Proposal · MDS301 Machine Learning in Data Sciences · 27 October 2026
Team: Bui Minh Hoang, Tran Tuan Hung, Ho Minh Hieu (roll numbers: TBD) · Repository: github.com/minhhoangb/food-recipe-recommendation-system

1. Problem statement and users

Home cooks often have a handful of ingredients in the fridge and no idea what to make with them, so food is wasted or a meal is bought instead. Searching recipe sites by dish name does not help when the starting point is "*I have chicken thighs, rice and soy sauce*". Our system takes the ingredients a user has and returns the recipes that best use them, each with the ingredients the user already has (*matched*), those still needed (*missing*), the directions and a link to the source.

Who benefits: people cooking at home who want to use up what they have (students, families, anyone reducing food waste), and recipe or grocery apps that could embed the API. The scope is English recipes and ingredient-list queries; dietary, allergy or nutrition guarantees are out of scope and the interface must say so (no allergy filtering is claimed).

2. ML formulation

Task type: content-based recommendation framed as ranked retrieval (NLP). Given a set of ingredient names q, rank the 50,000 candidate recipes so the most usable ones come first. There are no users, ratings or clicks in the data, so collaborative filtering and matrix factorisation do not apply; there is no fixed label per recipe, so plain classification or regression do not fit either. Each recipe is an item described by text (title, ingredient list), which makes this a content-based, text-similarity problem.

- **Classical models (scikit-learn):** TF-IDF + cosine similarity (E01; E02 = TF-IDF tuned with 5-fold group cross-validation), BM25 (E03, rank-bm25) and LSA, i.e. TruncatedSVD over TF-IDF (dimensionality reduction, unsupervised).
- **Deep learning:** the transformer bi-encoder `sentence-transformers/all-MiniLM-L6-v2`, first pretrained (E04: ingredients only, E05: title + ingredients), then fine-tuned on our training pairs with MultipleNegativesRankingLoss (E07-E09, including a regularisation ablation).
- **Labels:** (1) *synthetic queries*: 2-6 ingredients sampled from a recipe, at least one ingredient hidden and at least one non-common ingredient kept; the origin recipe is the known positive, so Recall@k / MRR@10 measure an *origin-recipe retrieval proxy*; (2) *manual graded labels* (0/1/2) on 40 natural English queries (20 development, 20 blind final), pooled across models, for nDCG@10.
- **Leakage control:** recipes sharing an ingredient set or a title form a group; groups never cross the train / validation / test split. Vectorisers, IDF and fine-tuning see train only; test is read once, after the configuration is frozen.

3. Dataset

RecipeNLG (Bien et al., INLG 2020), recipenlg.cs.put.poznan.pl (also mirrored on Kaggle). One CSV of 2.29 GB with 2,231,142 recipes and 7 columns: index, `title`, `ingredients` (raw lines), `directions`, `link`, `source` (`Gathered` or `Recipes1M`) and `NER` (ingredient names extracted by the authors). The raw file stays unmodified in `data/raw/` (sha256 `f376ee0b…`); it is not committed and is fetched or rebuilt with the documented scripts.

Subset (within the course limit of 2,000-500,000 samples). A seeded random draw (seed 42) over all raw ids, because the file is sorted by source and its first rows would be biased. Of 50,360 recipes examined, 180 were dropped as exact duplicates and 180 for having fewer than 2 or more than 40 normalised ingredients, giving **50,000 recipes** (`main`; the first 5,000 form a `pilot` profile).

Item (main)	Value	Item (main)	Value
Recipes / groups	50,000 / 39,869	Train / validation / test recipes	35,896 / 3,970 / 10,134
Singleton groups / largest group	92.7% / 141	Synthetic queries train / val / test	5,000 / 500 / 1,000
Ingredients per recipe (mean, median, range)	8.25, 8, 2-37	Source mix Gathered / Recipes1M	73.7% / 26.3%

Features. 7 raw columns; models use the normalised ingredient list (vocabulary of 11,901 names fitted on train), the title for E05, and two engineered features (ingredient count, mean ingredient rarity fitted on train) plus query-time matched/missing counts. How the course's "8-200 features" rule applies to text input: TBD with the instructor.

Known data quality issues (raw scan and a manual review of 100 recipes, `reports/data_audit.md`):

- **Noisy ingredient names:** 36 of 100 reviewed recipes have at least one wrong or truncated normalised ingredient (e.g. NER keeps "*virgin*" and loses "*coconut oil*"; brand fragments like "*dew*").
- **Missing entries:** NER has fewer entries than the raw lines in 20.2% of recipes; 573 raw recipes have an empty NER list (raw-line fallback used); 1 null title.
- **Duplicates and near-duplicates:** 1,245 exact duplicates in the raw file (0.06%); near-duplicates are grouped, not removed, so they cannot leak across splits.
- **Skew:** 60% of distinct names occur in one recipe only, while salt appears in 43.5% of recipes. Links lack a scheme (the pipeline adds `https://`).

Licence: not assigned. We have not verified RecipeNLG's terms of use and assume no licence. The repository does not redistribute the raw data; the terms must be checked before any redistribution or public deployment of recipe text (owner: Hoang, before 1 November).

4. Success criteria

#	Criterion (deployed model unless stated)	Target	Status today
1	Recall@10 on the synthetic **test** split (1,000 queries, read once after freezing)	≥ 0.85	Test: TBD. Validation (500 q): best system BM25 0.910, TF-IDF 0.826, MiniLM pretrained 0.576
2	MRR@10 on the synthetic test split	≥ 0.70	Test: TBD. Validation: BM25 0.758, TF-IDF 0.657, MiniLM pretrained 0.434
3	Pooled nDCG@10 on the 20 blind manual queries, with Cohen's κ on the ≥ 20% double-labelled pool	nDCG@10 ≥ 0.60, κ ≥ 0.60	TBD (guideline and queries ready; labelling in frames 3-4)
4	API latency for `/predict`, top_k = 10, on one CPU core (p95)	≤ 300 ms	TBD (benchmark in frame 3)
5	Fine-tuned MiniLM vs. pretrained on validation MRR@10 (same format and corpus), paired test	better at $p < 0.01$	TBD; if not met, it is reported and the better model is deployed

Validation numbers: MLflow experiment `recipe-retrieval`, corpus sha256 1f1f4ce5...; run ids listed in `reports/comparison.csv`. TF-IDF tuning (GroupKFold(5), 24 grid points) kept the defaults: CV MRR@10 0.690 ± 0.006. Synthetic labels have one known positive per query, so criteria 1-2 are a proxy and criterion 3 checks real usefulness.

5. Project plan

Work is split into 48-hour frames; every task runs on its own feature branch and merges into `dev` through a reviewed pull request (Hoang reviewed by Hieu, Hung by Hoang, Hieu by Hung). Configuration freeze and the single test run: 1 November. Hard deadline: 7 November.

Week / frame	Hoang: data, baselines, evaluation, report	Hung: embeddings, API, UI, Docker	Hieu: fine-tuning, MLflow, monitoring
Week 1 (19-25 Oct) Frame 1: 23-25 Oct	Data pipeline, group split, synthetic queries, EDA and data audit; TF-IDF / BM25 / LSA baselines and ranx evaluator on pilot (done)	MiniLM encoder, dense retriever, E04/E05 on pilot (done)	Training pairs, Colab fine-tuning notebook with checkpoint/resume, CPU export (done)
Week 2 (26-31 Oct) Frame 2: 26-27 Oct	E02 CV tuning, validation comparison of E01-E05 with significance tests, this proposal, manual query set and guideline	Main 50k index, recommend() with matched/missing, E06 rerank ablation, FastAPI, Docker	Main fine-tuning (2 configs), regularisation ablation, MLflow registry (staging)
Frame 3: 28-29 Oct	Blind dev pool and labels, error analysis (≥ 20 cases), report part 1, tests and coverage	Web UI, latency/RAM benchmark, bundle selection by config, dev labels	Drift monitoring, one-command `train.py`, registry aliases and rollback, dev labels
Frame 4: 30-31 Oct	Freeze, single test run (E10), final labels and kappa, Master Comparison Table, report part 2	Docker and README from a clean clone, demo runbook, smoke tests, final labels	FT vs. pretrained analysis, monitoring on real API logs, MLflow evidence, DL/MLOps report, final labels
Week 3 (1-7 Nov) Frame 5: 1-2 Nov	Merge the 15-25 page report, slides on problem/data/models/results, README audit, clean-clone check, release tag	Deployment slides, live demo rehearsal, backup demo video	DL/MLOps slides, Q&A notes, full rehearsal (20 min + 10 min Q&A)
3-7 Nov	Buffer only: bug fixes, rehearsal, submission. No new features.		

6. Risks

- **Proxy labels** reward finding one recipe and may underrate good alternatives → manual graded labels (criterion 3).
- **Fine-tuning may not beat lexical baselines** (on validation BM25 already leads) → reported as is; the deployed model is chosen on validation, not on expectations.
- **Free Colab GPU limits** → checkpoint/resume and a CPU path with fewer steps.
- **Dependency conflicts** between training and serving packages → pinned, separately tested requirements.